

Human-in-the-loop sudo approval gate for autonomous LLM agents

A sudo approval plugin running inside KubeVirt VMs can intercept every privileged command, forward it over a Unix socket to a local Go daemon, which relays it via NATS to a FastAPI orchestrator for human approval — all using the purpose-built approval plugin API introduced in sudo 1.9.0. Ubuntu 24.04 ships sudo 1.9.15p5, [\(Linux Mint Forums\)](#) fully supporting this API. The architecture is fail-closed by design: any failure in the plugin, daemon, NATS, or orchestrator results in command denial. This report provides the complete technical blueprint — from exact C struct definitions and return codes through daemon architecture to orchestrator schema and security hardening.

1. The sudo approval plugin API: a deep technical dive

Plugin types and why approval is the correct choice

The sudo plugin API (defined in `sudo_plugin.h`) supports `Sudo` five plugin types. [\(Linux Man Pages\)](#) Understanding each clarifies why the **approval plugin** is the only correct choice for command gating.

The **policy plugin** (`type=1`) determines whether a user may run a command [\(Sigma-star\)](#) — but only one can be active at a time, and replacing sudoers is inadvisable. The **I/O plugin** (`type=2`) logs and filters session I/O; [\(Sigma-star\)](#) its `open()` can reject commands, but this overloads a logging API for authorization, `Sudo` forces PTY allocation, and keeps the plugin alive for the entire command execution. The **audit plugin** (`type=3`) receives accept/reject/error events for logging [\(Sigma-star\)](#) but cannot influence the decision. The **group provider plugin** handles external group lookups during sudoers evaluation. [\(Man Pages\)](#)

The **approval plugin** (`type=4`, `SUDO_APPROVAL_PLUGIN`) was introduced in **sudo 1.9.0 (May 2020)** specifically to add authorization layers without replacing sudoers. [\(Linux Man Pages\)](#) [\(Opensource.com\)](#) It runs after the policy plugin succeeds, `Sudo` [\(Opensource.com\)](#) has a short lifecycle (open → check → close, all before execution begins), and its `check()` function **can block synchronously** — sudo imposes no timeout. Multiple approval plugins can be registered; **all must return success** for the command to proceed. `Sudo` This is precisely the semantics needed for a human approval gate.

Struct definition and function signatures

The approval plugin struct from `sudo_plugin.h`:

```
c
```

```

struct approval_plugin {
#define SUDO_APPROVAL_PLUGIN    4
    unsigned int type;          /* always SUDO_APPROVAL_PLUGIN */
    unsigned int version;       /* always SUDO_API_VERSION */
    int (*open)(unsigned int version, sudo_conv_t conversation,
                sudo_printf_t sudo_printf, char * const settings[],
                char * const user_info[], int submit_optind,
                char * const submit_argv[], char * const submit_envp[],
                char * const plugin_options[], const char **errstr);
    void (*close)(void);
    int (*check)(char * const command_info[], char * const run_argv[],
                 char * const run_envp[], const char **errstr);
    int (*show_version)(int verbose);
};

```

The **open()** function receives the full context of the sudo invocation: `settings[]` contains sudo command-line options (`runas_user`, `progname`); [Linux Man Pages](#) `user_info[]` provides the invoking user's identity (`user`, `uid`, `gid`, `groups`, `cwd`, `tty`, `host`, `pid`, `ppid`); [man7](#) `submit_argv[]` and `submit_envp[]` carry the original invocation and environment; `plugin_options[]` passes arguments from the `Plugin` line in `/etc/sudo.conf`. [Linux Man Pages](#) All are NULL-terminated arrays of "name=value" strings. [Sudo +2](#) Returns **1** on success, **0** on failure, **-1** on error, **-2** on usage error. [man7](#)

The **check()** function is the core gate. It receives `command_info[]` (fully qualified command path, `runas_uid`, `runas_gid`, `cwd`), and dozens more fields produced by the policy plugin), `run_argv[]` (the exact argument vector for `execve()`), and `run_envp[]` (the sanitized execution environment). [Sudo](#) Return codes:

Value	Meaning
1	Approved — command proceeds
0	Denied — command is rejected
-1	General error — command denied
-2	Usage error — sudo prints usage, exits

The **close()** function takes no arguments — unlike I/O or policy plugins, the approval plugin's lifecycle ends before command execution. It is called immediately after `check()` or `show_version()` returns. [Linux Man Pages](#)

Execution order and lifecycle

The definitive execution order within sudo is: **(1)** policy plugin `open()` → `check_policy()`, **(2)** if

policy approves → approval plugin(s) `open()` → `check()` → `close()`, (3) if all approval plugins approve → I/O plugin(s) `open()`, (4) audit plugin `accept()`, (5) command execution. If any approval plugin returns non-1, the command is denied and audit plugins receive a reject event typed as `SUDO_APPROVAL_PLUGIN`. [Linux Man Pages](#)

The plugin runs as root (uid 0) because sudo is a setuid-root binary. All plugin code executes in the sudo process before any uid/gid transition to the target user. This means the plugin has full privileges to open Unix sockets, read `/proc`, and perform system calls — but also that bugs in the plugin code carry root-level risk.

Blocking behavior and timeouts

Sudo imposes no timeout on approval plugin execution. The `check()` function blocks the sudo process until it returns. This is confirmed by the absence of any timeout documentation and by the API design: `event_alloc()` (for accessing sudo's event loop) is not part of the `approval_plugin` struct. The plugin must implement its own timeout using `SO_RCVTIMEO` on sockets, `alarm()` `SIGALRM`, or POSIX timers. A **300-second default timeout** with `SO_RCVTIMEO` is recommended — long enough for human review, short enough to avoid indefinite hangs.

Configuration in `/etc/sudo.conf`

```
# Standard sudoers policy (unchanged)
Plugin sudoers_policy sudoers.so
Plugin sudoers_io sudoers.so
Plugin sudoers_audit sudoers.so

# Custom approval gate – options passed as plugin_options[] to open()
Plugin sudo_gate_approval sudo_gate.so \
    socket_path=/run/sudo-gated/sudo-gated.sock \
    timeout=300 \
    fail_mode=deny
```

The syntax is `Plugin <symbol_name> <plugin_path> [options...]`. Relative paths resolve from the plugin directory (`/usr/lib/sudo/` on Ubuntu). [Arch Linux Man Pages](#) [Linux Man Pages](#) The shared object **must be owned by uid 0 and only writable by its owner** [Man Pages](#) [Linux Man Pages](#) — sudo enforces this check and refuses to load non-compliant plugins with a fatal error. Options after the `.so` path are split on whitespace and passed as the `plugin_options[]` array to `open()`. [Man Pages](#) [Linux Man Pages](#)

2. Skeleton C implementation with Unix socket communication

The following skeleton implements a complete approval plugin that connects to a local daemon over a Unix domain socket, sends a JSON request with command details, and blocks until

receiving an approve/deny response.

c

```

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/socket.h>
#include <sys/un.h>
#include <time.h>
#include "sudo_plugin.h"
#include "cJSON.h"

#define DEFAULT_SOCKET  "/run/sudo-gated/sudo-gated.sock"
#define DEFAULT_TIMEOUT 300
#define BUF_SIZE        8192

static sudo_conv_t  conv_fn;
static sudo_printf_t printf_fn;
static const char   *socket_path = DEFAULT_SOCKET;
static int          timeout_sec = DEFAULT_TIMEOUT;
static int          fail_open   = 0;

/* Saved from open() for use in check() */
static const char *invoking_user = NULL;
static const char *invoking_host = NULL;
static const char *invoking_tty  = NULL;
static const char *invoking_cwd  = NULL;

/* Helper: find value for key in name=value array */
static const char *find_value(char * const arr[], const char *key) {
    if (!arr) return NULL;
    size_t klen = strlen(key);
    for (int i = 0; arr[i]; i++) {
        if (strncmp(arr[i], key, klen) == 0 && arr[i][klen] == '=')
            return arr[i] + klen + 1;
    }
    return NULL;
}

/* Connect to daemon with timeout */
static int connect_to_daemon(void) {
    int fd = socket(AF_UNIX, SOCK_STREAM, 0);
    if (fd < 0) return -1;

    struct timeval tv = { .tv_sec = timeout_sec, .tv_usec = 0 };
    setsockopt(fd, SOL_SOCKET, SO_RCVTIMEO, &tv, sizeof(tv));
    setsockopt(fd, SOL_SOCKET, SO_SNDTIMEO, &tv, sizeof(tv));

```

```

    struct sockaddr_un addr = { .sun_family = AF_UNIX };
    strncpy(addr.sun_path, socket_path, sizeof(addr.sun_path) - 1);

    if (connect(fd, (struct sockaddr *)&addr, sizeof(addr)) < 0) {
        close(fd);
        return -1;
    }
    return fd;
}

static int sudo_gate_open(unsigned int version, sudo_conv_t conversation,
    sudo_printf_t plugin_printf, char * const settings[],
    char * const user_info[], int submit_optind,
    char * const submit_argv[], char * const submit_envp[],
    char * const plugin_options[], const char **errstr)
{
    conv_fn    = conversation;
    printf_fn  = plugin_printf;

    /* Parse plugin options from sudo.conf */
    for (int i = 0; plugin_options && plugin_options[i]; i++) {
        const char *v;
        if ((v = find_value(&plugin_options[i], "socket_path")))
            socket_path = v; /* pointer into sudo-owned memory, valid until close */
        else if ((v = find_value(&plugin_options[i], "timeout")))
            timeout_sec = atoi(v);
        else if ((v = find_value(&plugin_options[i], "fail_mode")))
            fail_open = (strcmp(v, "open") == 0);
    }

    /* Cache user info for check() */
    invoking_user = find_value(user_info, "user");
    invoking_host = find_value(user_info, "host");
    invoking_tty  = find_value(user_info, "tty");
    invoking_cwd  = find_value(user_info, "cwd");

    return 1;
}

static void sudo_gate_close(void) {
    /* No persistent resources to clean up */
}

static int sudo_gate_check(char * const command_info[],
    char * const run_argv[], char * const run_envp[],
    const char **errstr)
{

```

```

int fd, ret = 0;
char buf[BUF_SIZE];

/* Build JSON request */
const char *command = find_value(command_info, "command");
const char *runas_user = find_value(command_info, "runas_user");

cJSON *req = cJSON_CreateObject();
cJSON_AddStringToObject(req, "command", command ? command : "unknown");
cJSON_AddStringToObject(req, "user", invoking_user ? invoking_user : "unknown");
cJSON_AddStringToObject(req, "runas_user", runas_user ? runas_user : "root");
cJSON_AddStringToObject(req, "host", invoking_host ? invoking_host : "unknown");
cJSON_AddStringToObject(req, "tty", invoking_tty ? invoking_tty : "");
cJSON_AddStringToObject(req, "cwd", invoking_cwd ? invoking_cwd : "");

/* Add argv array */
cJSON *argv_arr = cJSON_AddArrayToObject(req, "argv");
for (int i = 0; run_argv && run_argv[i]; i++)
    cJSON_AddItemToArray(argv_arr, cJSON_CreateString(run_argv[i]));

char *json_str = cJSON_PrintUnformatted(req);

/* Connect to daemon */
fd = connect_to_daemon();
if (fd < 0) {
    if (fail_open) {
        printf_fn(SUDO_CONV_ERROR_MSG,
            "sudo-gate: daemon unavailable, allowing (fail-open)\n");
        ret = 1;
    } else {
        *errstr = "Approval daemon is not running";
        printf_fn(SUDO_CONV_ERROR_MSG, "sudo-gate: %s\n", *errstr);
    }
    goto cleanup;
}

/* Send request with length prefix (4 bytes big-endian + JSON) */
uint32_t len = htonl(strlen(json_str));
if (send(fd, &len, 4, 0) != 4 || send(fd, json_str, strlen(json_str), 0) < 0) {
    *errstr = "Failed to send request to daemon";
    close(fd);
    goto cleanup;
}

printf_fn(SUDO_CONV_INFO_MSG,
    "sudo-gate: Waiting for approval (timeout %ds)...\n", timeout_sec);

```

```

/* Read response (blocking, with SO_RCVTIMEO) */
uint32_t resp_len;
ssize_t n = recv(fd, &resp_len, 4, MSG_WAITALL);
if (n != 4) {
    *errstr = "Approval request timed out or daemon disconnected";
    printf_fn(SUDO_CONV_ERROR_MSG, "sudo-gate: %s\n", *errstr);
    close(fd);
    goto cleanup;
}
resp_len = ntohl(resp_len);
if (resp_len >= BUF_SIZE) resp_len = BUF_SIZE - 1;

n = recv(fd, buf, resp_len, MSG_WAITALL);
close(fd);
if (n <= 0) {
    *errstr = "Empty or failed response from daemon";
    goto cleanup;
}
buf[n] = '\0';

/* Parse response */
cJSON *resp = cJSON_Parse(buf);
if (!resp) {
    *errstr = "Malformed response from daemon";
    goto cleanup;
}

cJSON *approved = cJSON_GetObjectItem(resp, "approved");
if (cJSON_IsTrue(approved)) {
    ret = 1; /* APPROVED */
} else {
    cJSON *reason = cJSON_GetObjectItem(resp, "reason");
    const char *r = cJSON_IsString(reason) ? reason->valstring : "denied";
    printf_fn(SUDO_CONV_ERROR_MSG, "sudo-gate: Request denied: %s\n", r);
    *errstr = "sudo request denied by operator";
}
cJSON_Delete(resp);

cleanup:
free(json_str);
cJSON_Delete(req);
return ret;
}

static int sudo_gate_show_version(int verbose) {
    printf_fn(SUDO_CONV_INFO_MSG, "sudo-gate approval plugin version 0.1.0\n");
    return 1;
}

```

```

}

/* Exported symbol referenced in sudo.conf */
sudo_dso_public struct approval_plugin sudo_gate_approval = {
    SUDO_APPROVAL_PLUGIN,
    SUDO_API_VERSION,
    sudo_gate_open,
    sudo_gate_close,
    sudo_gate_check,
    sudo_gate_show_version
};

```

Compilation and installation:

```

bash

# Vendor cJSON (single-file library, no runtime dependency)
wget https://raw.githubusercontent.com/DaveGamble/cJSON/master/cJSON.{c,h}

# Get sudo_plugin.h from source
apt-get source sudo # provides sudo-1.9.15p5/include/sudo_plugin.h

# Build shared library
gcc -shared -fPIC -Wall -Wextra -O2 \
    -I./include \
    -o sudo_gate.so sudo_gate.c cJSON.c

# Install (must be root-owned, owner-writable only)
sudo install -o root -g root -m 0644 sudo_gate.so /usr/lib/sudo/
sudo chmod +i /usr/lib/sudo/sudo_gate.so

```

No linking against libsudo is needed — the plugin API is header-only, with function pointers provided at runtime by sudo.

3. Reference implementations and prior art

Square's `sudo_pair` (github.com/square/sudo_pair, ~1.3k stars) is the most relevant existing project. ([GitHub](#)) Written in Rust with C FFI, it implements an I/O plugin ([Opensource.com](#)) (not approval plugin — it predates sudo 1.9) that requires a second human to approve and monitor privileged sessions. ([GitHub](#)) It creates per-session Unix sockets in `/var/run/sudo_pair/`, owned by the target user's group, so only someone who can sudo to that user can approve. ([GitHub](#)) The architecture — plugin → Unix socket → approval agent — directly parallels the proposed design.

Sudo's official `sample_approval.c` ships in the source tree (Apple Open Source) at `plugins/sample_approval/`. It implements a "business hours" approval plugin that denies commands outside Mon–Fri 9–17. (Apple Open Source) While trivial, it demonstrates the exact struct layout, registration pattern, and return code conventions. The sudo project also ships a Python example (`example_approval_plugin.py`) using the Python plugin bindings, (Sudo) (Sudo) though Ubuntu 24.04's sudo is not built with `--enable-python`.

Sigma Star's "sudo-shame" plugin (blog.sigma-star.io, July 2025) demonstrates a C audit plugin that forks an external script from within the plugin, (Sigma-star) showing patterns for privilege dropping (`setuid`)(`setgid`) and pipe-based IPC from plugin context. The patterns are directly transferable to an approval plugin that needs to communicate with an external daemon.

No existing open-source project implements the exact "approval plugin → daemon → external orchestrator" pattern proposed here. This appears to be novel architecture, though it follows naturally from the approval plugin API's design intent.

4. Daemon architecture: Go with NATS request/reply

Why Go over Rust for sudo-gated

Go is the clear choice for this specific daemon. The NATS ecosystem is Go-native — the `nats.go` client is v1.x+, battle-tested, and the most mature client across all languages. Rust's `async-nats` remains on **0.x versioning**. systemd integration in Go is turnkey via `coreos/go-systemd/v22` (GitHub) (GitHub) (pure Go, no cgo), providing socket activation, `sd_notify`, and watchdog support. (Go Packages) A fully static binary is trivially produced with `CGO_ENABLED=0` — no musl toolchain needed. (Dasroot) Most critically, **goroutine-per-connection is the ideal concurrency model** here: each sudo invocation connects, the goroutine blocks on `nc.Request()` waiting for human approval (potentially minutes), and returns. This is natural in Go but awkward in async Rust where long-lived blocking futures complicate the runtime.

Rust's advantages — memory safety beyond GC, smaller binaries (~3–8 MB vs ~10–15 MB) — are irrelevant. This is a low-frequency daemon (sudo requests per minute, not per second), not a performance-critical proxy. Development speed is **3–5x faster in Go** for this scope.

(Bitfield Consulting)

Communication: core NATS, not JetStream

Use core NATS request/reply, not JetStream. The sudo approval flow is inherently synchronous — the plugin blocks waiting for a response. JetStream's persistent streaming semantics are wrong here: a stale approval from 10 minutes ago should never execute. Core NATS provides "at-most-once" delivery with `nc.Request(subject, data, timeout)`, built-in timeout handling, and immediate `ErrNoResponders` detection (NATS 2.2+) if no orchestrator is subscribed.

Subject naming follows the existing NATS hierarchy: `sudo.request.{vm_id}.{job_id}`. The orchestrator subscribes with wildcards (`sudo.request.>`) to receive all requests, or

`sudo.request.{specific_vm}.>` for filtered monitoring.

```
go

// Daemon forwards to orchestrator, blocks for human decision
msg, err := nc.Request(
    fmt.Sprintf("sudo.request.%s.%s", vmID, jobID),
    requestPayload,
    5*time.Minute, // human approval timeout
)
if err == nats.ErrNoResponders {
    return DenyResponse("orchestrator unavailable")
}
if err == nats.ErrTimeout {
    return DenyResponse("approval timed out")
}
```

The `nats.go` client provides **automatic reconnection** with configurable retry, jitter, and buffer size. [Go Packages](#) During NATS outages, requests fail immediately with a deny response — the plugin receives this, prints an error, and sudo denies the command.

Systemd integration and security hardening

The daemon should use **systemd socket activation** (`sudo-gated.socket` unit) so systemd creates the socket at `/run/sudo-gated/sudo-gated.sock` with correct permissions (`SocketMode=0660`, `SocketUser=root`, `SocketGroup=sudo-gated`). This enables zero-downtime restarts — new connections queue in the kernel's listen backlog while the daemon restarts. [bernat](#) [Opointer](#) The service unit uses `Type=notify` with `sd_notify` readiness signaling [Go Packages](#) after NATS connection establishment, and `WatchdogSec=30` for liveness monitoring. [GitHub](#)

The service file should apply aggressive systemd hardening:

- `NoNewPrivileges=yes`, `ProtectSystem=strict`, `ProtectHome=yes`, `PrivateTmp=yes` [GitHub](#)
- `MemoryDenyWriteExecute=yes` [Ruderich](#) (safe for Go — no JIT)
- `SystemCallFilter=@system-service` (whitelist standard service syscalls)
- `CapabilityBoundingSet=` (empty — no capabilities needed)
- `RestrictAddressFamilies=AF_UNIX AF_INET AF_INET6` [GitHub](#) (UDS + NATS/HTTP)
- `ReadWritePaths=/run/sudo-gated` (socket directory only)

Rate limiting and graceful lifecycle

A **token bucket** rate limiter ([golang.org/x/time/rate](#)) at **5 requests/minute with burst of 3** prevents an LLM agent from flooding sudo invocations. Token bucket is preferred over fixed-

window because it allows legitimate bursts (e.g., a build script running 3 `apt-get install` commands sequentially) while enforcing a sustained rate ceiling.

For graceful shutdown: `signal.NotifyContext` converts SIGTERM into context cancellation, `sync.WaitGroup` tracks in-flight goroutines, `freeCodeCamp` a 30-second grace period allows pending approvals to complete, and `nc.Drain()` flushes NATS buffers before exit.

5. Orchestrator and UI integration

Database schema for approval requests

```
sql

CREATE TYPE sudo_request_status AS ENUM (
    'pending', 'approved', 'denied', 'expired', 'timeout'
);

CREATE TABLE sudo_approval_requests (
    id                UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    job_id            UUID NOT NULL REFERENCES jobs(id) ON DELETE CASCADE,
    agent_id          UUID NOT NULL REFERENCES agents(id) ON DELETE CASCADE,
    vm_name           VARCHAR(255) NOT NULL,
    command           TEXT NOT NULL,
    arguments         TEXT[] DEFAULT '{}',
    environment       JSONB DEFAULT '{}',
    working_directory TEXT,
    requesting_user    VARCHAR(255) NOT NULL,
    target_user       VARCHAR(255) NOT NULL DEFAULT 'root',
    status            sudo_request_status NOT NULL DEFAULT 'pending',
    requested_at      TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    decided_at        TIMESTAMPTZ,
    decided_by        VARCHAR(255),
    decision_reason    TEXT,
    auto_approved     BOOLEAN NOT NULL DEFAULT FALSE,
    ttl_seconds       INTEGER NOT NULL DEFAULT 300,
    expires_at        TIMESTAMPTZ GENERATED ALWAYS AS
        (requested_at + (ttl_seconds || ' seconds')::INTERVAL) STORED,
    metadata          JSONB DEFAULT '{}'
);
```

Critical indexes: a partial index on `((status, requested_at DESC) WHERE status = 'pending')` serves the hot path (UI polling and SSE), `((job_id, requested_at DESC))` for job-scoped views, and `((expires_at) WHERE status = 'pending')` for the expiration sweep. Expiration is best handled by an **asyncio background task** inside FastAPI's lifespan context, running every 10–30 seconds, which allows coupling expiration with SSE event emission. A belt-and-suspenders

read-time filter (`WHERE expires_at > NOW()`) prevents stale pending requests from appearing in API responses.

Real-time push via SSE

Since SSE is already used for builder chat, **extend the SSE infrastructure** rather than introducing WebSocket. Sudo approval notifications are strictly server-to-client push; `WebSocket` approve/deny actions are standard HTTP POST. A dedicated endpoint at `/api/v1/sudo/events` uses FastAPI's built-in `EventSourceResponse`. `FastAPI` The Angular client subscribes via `EventSource` with native auto-reconnection `System Design School` and `Last-Event-ID` resume support.

Approval UX drawn from industry patterns

GitHub Actions shows environment name, workflow, triggering actor, and commit SHA with one-click "Approve and deploy" / "Reject" buttons. `GitHub` Terraform Cloud displays the full plan diff with resource change counts as a risk indicator. For the sudo gate, the approval card should display:

- **Full command string** (syntax-highlighted, monospace)
- **Risk badge:** ● Low (read-only commands), ● Medium (package installs), ● High (permission changes), ⊖ Critical (shell spawns, destructive ops)
- **Context:** requesting user → target user, VM name, job link, working directory
- **Countdown timer** showing time remaining before expiry
- **One-click approve/deny** with optional reason field (required for deny)

Auto-approval with glob patterns

Use `fnmatch` (Python's `fnmatch.fnmatch()`) for pattern matching — it is safe (no backtracking, no ReDoS risk) and sufficient for command whitelisting. Rules are stored in a database table with priority ordering, evaluated highest-priority-first:

- **Always approve** (low risk): `apt-get install *-dev`, `cat *`, `ls *`, `systemctl status *`
- **Always deny** (critical risk): `rm -rf /`, `dd *`, `mkfs*`, `shutdown`, `reboot`
- **Always require review:** `bash *`, `sh *`, `python* *`, `chmod *`, `chown *`

Commands containing pipes (`|`), redirects (`>`), or chaining operators (`;`, `&&`, `||`) should **never be auto-approved** — these allow arbitrary execution chains that bypass per-command approval. The auto-approval engine must evaluate against the daemon-verified command (from `/proc` inspection), not the agent's self-reported payload.

6. Security analysis and attack surface

Plugin layer: the agent cannot escape the gate

The agent user (`agent-host`) cannot modify `/etc/sudo.conf` (root:root, mode 0644) or replace the plugin `.so` in `/usr/lib/sudo/` (root:root, mode 0755 directory). Sudo's own loader verifies that the shared object is root-owned and owner-writable-only, refusing to load non-compliant files. [Linux Man Pages](#) Apply `chattr +i` to `/etc/sudo.conf`, `/etc/sudoers`, and the plugin `.so` for defense-in-depth. Linux's SUID execution strips `LD_PRELOAD` and `LD_LIBRARY_PATH`, preventing library injection. [Medium](#) The sudoers `env_reset` default and `secure_path` setting create a clean execution environment. [Sudo](#) [Ubuntu](#)

Daemon layer: kernel-enforced peer verification

The Unix socket uses filesystem namespace (not abstract namespace, which has no permissions). Socket permissions of **0660 root:sudo-gated** restrict connections. [DEV Community](#) The daemon should use `SO_PEERCRED` to obtain the connecting process's PID, UID, and GID from the kernel — these values **cannot be forged**. Cross-referencing with `/proc/{pid}/exe` confirms the connection originates from the real sudo binary, not a spoofed process.

TOCTOU mitigation is critical: an approval for `cat /etc/passwd` could theoretically be replayed or the target file changed between approval and execution. Each request carries a unique UUID nonce; approvals bind to the exact (command, arguments, cwd, nonce) tuple. The daemon should compute a **SHA-256 hash** of the command binary at request time and include it in the approval payload.

Orchestrator layer: NATS authorization and input sanitization

NATS authentication via NKey or JWT credentials restricts each VM daemon to publishing only on its own `sudo.request.{vm_id}.*` subjects. Command strings stored in PostgreSQL and rendered in Angular must be sanitized: strip ANSI escape sequences, Unicode bidirectional control characters (U+202A-U+202E, U+2066-U+2069), and rely on Angular's default template escaping (never use `[innerHTML]` for command display).

RBAC for approvers: `sudo_admin` (full access), `sudo_approver` (approve/deny for assigned jobs), `sudo_viewer` (read-only audit). Self-approval prevention ensures the user who launched the job cannot approve its sudo requests. [GitHub](#)

Failure mode analysis: fail-closed by default

Failure	Behavior	Outcome
Plugin <code>.so</code> cannot be loaded	sudo prints fatal error, exits	No commands run via sudo — lockout risk
<code>open()</code> returns non-1	<code>check()</code> not called, <code>close()</code> called	Command denied

Failure	Behavior	Outcome
<code>check()</code> crashes (segfault)	sudo receives error signal	Command denied
Daemon socket unavailable	<code>connect()</code> returns ENOENT/ECONNREFUSED	Command denied (configurable fail-open)
Daemon returns malformed JSON	<code>cJSON_Parse()</code> returns NULL	Command denied
NATS unavailable	<code>nc.Request()</code> returns error	Command denied
Orchestrator not subscribed	<code>ErrNoResponders</code> fires immediately	Command denied
No human responds within TTL	NATS request timeout	Command denied
Daemon restarts mid-request	UDS connection broken	Plugin receives error, command denied

The lockout risk when the plugin `.so` fails to load is the most dangerous failure mode. **Always test plugin changes with an existing root shell open**, and maintain console/recovery access to KubeVirt VMs via `virtctl console`.

7. Ubuntu 24.04 specifics and version matrix

Component	Version	Notes
sudo	1.9.15p5 (<code>(1.9.15p5-3ubuntu5)</code>)	Fully supports approval plugins (requires $\geq 1.9.0$)
systemd	255.4	Full support for all hardening directives listed above
Kernel	6.8	Full <code>SO_PEERCRED</code> support
GCC	14	Default compiler for plugin .so
Plugin directory	<code>/usr/lib/sudo/</code>	Differs from upstream <code>/usr/libexec/sudo/</code>
<code>sudo_plugin.h</code>	Not packaged separately	Obtain from <code>apt-get source sudo</code> or GitHub

Component	Version	Notes
cJSON	1.7.17 (<code>libcjson-dev</code> , universe)	Recommended: vendor single <code>.c/.h</code> file instead
Python	3.12	For FastAPI orchestrator

8. Build order and testing strategy

Phase 1 — Plugin stub (days 1-2): Build a minimal approval plugin that logs `check()` invocations to syslog and always returns 1 (approve). Install in `/usr/lib/sudo/`, register in `sudo.conf`, verify sudo still works. This validates the plugin loading, struct layout, and lifecycle without any external dependencies. **Test:** run `sudo ls` and confirm syslog shows the plugin's log line.

Phase 2 — Socket communication (days 3-4): Add Unix socket client code to `check()`. Build a trivial "echo daemon" (netcat or 20 lines of Go) that listens on the socket and replies `{"approved": true}`. Verify end-to-end flow. **Test:** run `sudo ls`, confirm daemon receives JSON, plugin receives response, command executes. Then test failure modes: kill daemon, verify deny. Set `SO_RCVTIMEO` to 5 seconds and verify timeout behavior.

Phase 3 — Go daemon with NATS (days 5-8): Build `sudo-gated` with Unix socket server, NATS `Request()` forwarding, systemd unit files, and rate limiting. Deploy alongside a NATS responder stub that auto-approves. **Test:** multiple concurrent `sudo` invocations, daemon restart under load, NATS disconnect/reconnect, rate limit exhaustion.

Phase 4 — Orchestrator integration (days 9-12): Add `sudo_approval_requests` table, NATS subscription in FastAPI lifespan, SSE endpoint, REST endpoints for approve/deny. Wire Angular UI cards. **Test:** end-to-end human approval flow — agent runs `sudo apt-get install nginx`, operator sees card in UI, clicks approve, command executes.

Phase 5 — Auto-approval and hardening (days 13-16): Implement auto-approval rules engine, RBAC, `SO_PEERCREDS` verification, `chattr +i` on plugin files, full systemd hardening, monitoring alerts. **Test:** auto-approved commands bypass UI, denied patterns reject immediately, `SO_PEERCREDS` rejects non-sudo connections, systemd restarts daemon on crash within 5 seconds.

Conclusion

The sudo approval plugin API is purpose-built for this exact use case — adding an authorization gate that blocks synchronously without replacing sudoers. The architecture of plugin → Unix socket → Go daemon → NATS → FastAPI orchestrator → human provides defense-in-depth with fail-closed semantics at every layer. Three non-obvious insights emerged from this research.

First, the approval plugin's `close()` function takes no arguments because the plugin lifecycle ends before command execution — a design choice that simplifies state management considerably. Second, `SO_PEERCRED` on Linux provides kernel-enforced, unforgeable peer identity on Unix sockets, which is the strongest available mechanism for verifying that only the real sudo process connects to the daemon. Third, the greatest operational risk is not security bypass but **lockout**: a broken plugin `.so` prevents all sudo usage on the VM, making console recovery access and careful deployment procedures essential.